Tecnologías de la Información

Universidad de Huelva

Tema 4

TADs Lineales Dinámicos

1. Implementación dinámica del TAD Lista.

El uso de tablas presenta ventajas ya vistas, pero también presentan desventajas:

- Reserva constante de memoria independientemente de su utilización y aprovechamiento.
- Limitación del número de elementos que puede contener.
- Imposibilidad de aumentar su capacidad.

La solución de estos problemas puede ser:

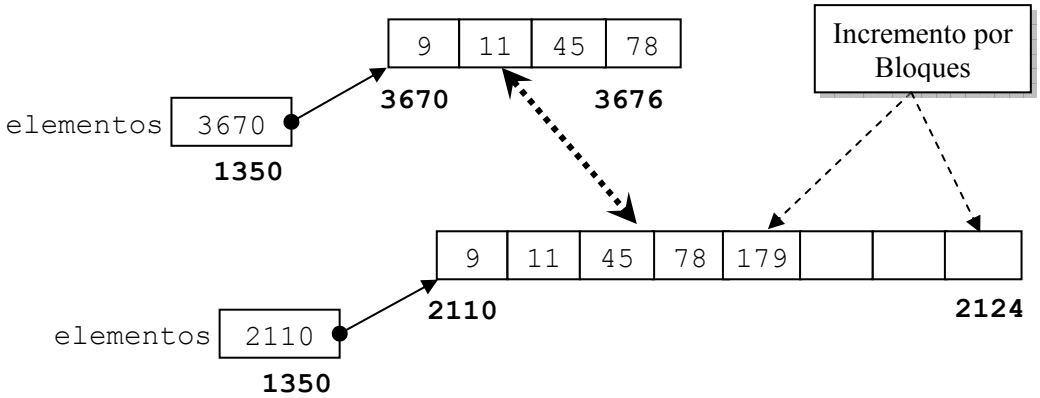
1. Evitar el problema que tienen las tablas de no poder crecer o decrecer utilizando para ellos **tablas dinámicas**.
   - **Ventaja**: el acceso a las tablas dinámicas es el mismo que a las tablas estáticas.
   - **Inconveniente**: cada vez que se modifica el tamaño de las tablas dinámicas, el S.O debe encontrar una zona en la memoria que se ajuste al nuevo tamaño pudiéndose dar el caso de que no exista.

2. Utilizando una estructura de datos encadenada mediante punteros que nos permite tener distribuidos en la memoria los elementos de la lista.
   - **Ventaja**: los nodos de la lista son almacenados en distintas partes de la memoria del ordenador. Para evitar su perdida, se utiliza en cada nodo un puntero que apunte al siguiente nodo.
   - **Inconveniente**: se necesita más memoria para almacenar el puntero al siguiente nodo de la lista.

1.1 Implementación con tablas dinámicas.

Tiene la posibilidad de aumentar y disminuir el tamaño de las tablas en tiempo de ejecución. Pasos:

1. Encontrar un espacio de memoria libre por parte del S.O.
2. Traspasar los datos de la zona original de memoria a la nueva zona que puede ser mayor o menor que la zona original.
3. Liberar el espacio de memoria antiguo para el uso por otro programa.



Ejemplo 1. *Representación gráfica del aumento de la capacidad de una tabla dinámica.*

Implementamos en C++ una lista donde el tipo de los elementos es real (**float**), haciendo uso de diseño modular y orientación a objetos.

```
// FICHERO TADLista.h
#include <iostream>
#define INCREMENTO 4
using namespace std;
class lista {
   float *elementos;  // elementos de la lista
   int  n;            // nº de elementos que tiene la lista
   int Tama;          // tamaño de la tabla en cada momento
 public:
   lista();  // constructor de la clase
   ~lista(); // destructor de la clase
   lista(float e);
   bool esvacia();
   int longitud();
   void anadirIzq(float e);
   void anadirDch(float e);
   void eliminarIzq();
   void eliminarDch();
   float observarIzq();
   float observarDch();
   void concatenar(lista l);
   bool pertenece(float e);
   void insertar(int i, float e);
   void eliminar(int i);
   void modificar(int i, float e);
   float observar(int i);
   int posicion(float e);
};

//FICHERO TADLista.cpp
# include "TADLista.h"
lista::lista()
{
  elementos=new float[INCREMENTO];
  if (elementos!=NULL)
  {
    Tama=INCREMENTO;
    n=0;
  }
  else
  {
    Tama=n=-1;
  }
}
```

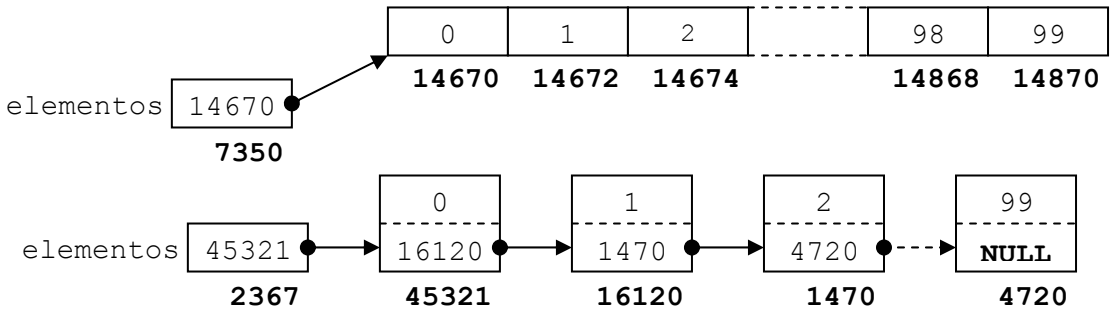
```
lista::~lista()
{
  if (elementos!=NULL)
    delete [] elementos;
  elementos=NULL;
  Tama=n=0;
}

void lista::insertar(int i, float e)
{
  int pos;
  if (n==Tama)
  {
    float *NuevaZona=new float[Tama+INCREMENTO];
    if (NuevaZona!=NULL)
    {
      for (int i=0;i<n; i++)
        NuevaZona[i]=elementos[i];
      Tama+=INCREMENTO;
      delete [] elementos;
      elementos=NuevaZona;
    }
  };

  if (n<Tama)
  {
    for (pos=n-1; pos>=i-1; pos--)
      elementos[pos+1]=elementos[pos];  // Desplazamiento
    elementos[i-1]=e;
    n++;
  }
}

void lista::modificar(int i, float e) {
  elementos[i-1]=e;
}

float lista::observar(int i) {
  return(elementos[i-1]);
}

bool lista::esvacia() {
  return (n == 0);
}
```

```
void lista::eliminar(int i)
{
  while (i<n)
  {
    elementos[i-1]=elementos[i]; // Desplazamiento
    i++;
  }
  n--;
  if (Tama-n>=INCREMENTO && Tama>INCREMENTO)
  {
    float *NuevaZona=new float[Tama-INCREMENTO];
    if (NuevaZona!=NULL)
    {
      Tama-=INCREMENTO;
      for (int i=0;i<Tama; i++)
        NuevaZona[i]=elementos[i];
      delete [] elementos;
      elementos=NuevaZona;
    };
  };
}

int lista::posicion(float e)
{
  int i=0;
  while (elementos[i]!=e && i < n)
    i++;
  return (i == n ? -1 : i+1);
}

int lista::longitud()
{
  return n;
}

lista::lista(float e)
{
  elementos=new float[INCREMENTO];
  if (elementos!=NULL) {
    Tama=INCREMENTO;
    n=1;
    elementos[0]=e;
  }
  else {
    Tama=n=-1;
  }
}
```

```
void lista::anadirIzq(float e)
{
  insertar(1, e);
}

void lista::anadirDch(float e)
{
  insertar(n+1, e);
}

void lista::eliminarIzq()
{
  eliminar(1);
}

void lista::eliminarDch()
{
  eliminar(n);
}

float lista::observarIzq()
{
  return(observar(1));
}

float lista::observarDch()
{
  return(observar(n));
}

void lista::concatenar(lista l)
{
  int lon = l.longitud();
  for (int i=1; i<=lon; i++)
    insertar(n+1, l.observar(i));
}

bool lista::pertenece(float e)
{
  return (posicion(e) != -1);
}
```

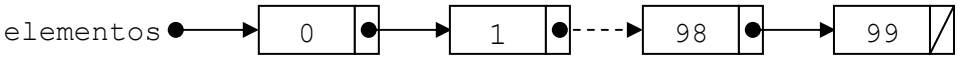
1.2 Implementación con nodos enlazados.

En vez de tener toda la información (nodos de la lista) agrupada en una misma zona de memoria (como las tablas estáticas y dinámicas), ahora se pretende dispersar cada elemento o nodo de la lista por cualquier zona de la memoria.



Ejemplo 2. *Representación gráfica de una tabla dinámica y de una lista dinámica enlazada.*

Para simplificar se va a obviar el poner las direcciones de memoria de manera que cuando exista un puntero existirá una flecha dirigida que indicará cuál es la variable puntero y cuál es la variable apuntada. Gráficamente el ejemplo anterior se simplifica como:



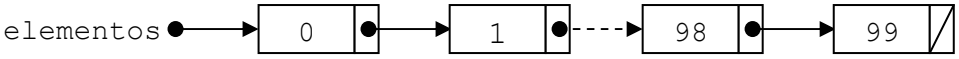
Ejemplo 3. *Representación gráfica de una lista de nodos.*

Inconvenientes: **No se puede** acceder de manera directa a cualquier nodo de la lista.
Se necesita más memoria para almacenar los distintos punteros.

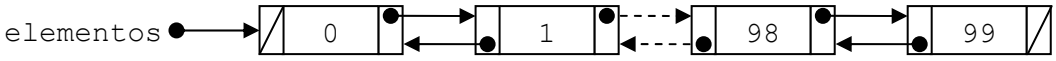
Ventajas: **Exige** menos movimiento de información cada vez que se modifica la lista.
Evita el problema de falta de espacio que se da con otras implementaciones.

Existen varios tipos de listas dinámicas enlazadas de datos que son:

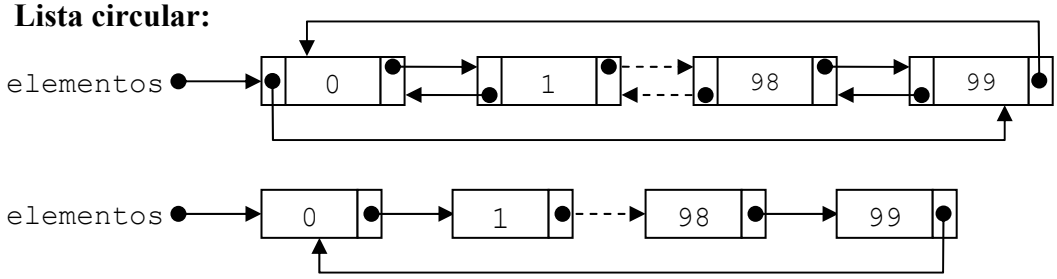
- **Lista simplemente enlazada**:



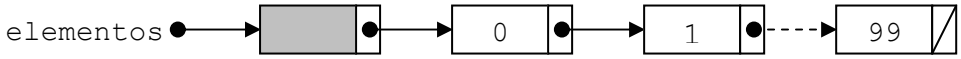
- **Lista doblemente enlazada**:



- **Lista circular:**

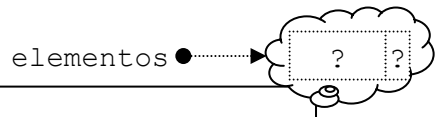


- **Lista con nodo cabecera**:



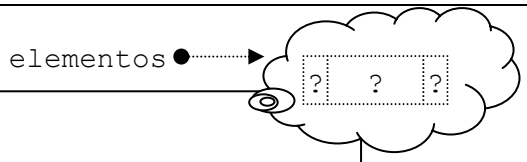
1.2.1 Operaciones básicas sobre listas enlazadas

Declaración




```
struct TNodo_Lista
{
  Tipo_elemento Datos;     //Datos a almacenar en cada nodo
  TNodo_Lista *Siguiente; //Puntero al siguiente nodo
};
TNodo_Lista *elementos; //Puntero al primer elemento
```

Ejemplo 3. *Declaración de una lista simplemente enlazada.*




```
struct TNodo_Lista
{
  Tipo_elemento  Datos;    //Datos a almacenar en cada nodo
  TNodo_Lista *Anterior;   //Puntero al nodo anterior
  TNodo_Lista *Siguiente; //Puntero al siguiente nodo
};
TNodo_Lista *elementos;      //Puntero al primer elemento
```

Ejemplo 4. *Declaración de una lista doblemente enlazada*

Creación

```
//La lista no contiene ningún nodo.
TNodo_Lista *elementos=NULL;
```

Ejemplo 5. *Creación de una lista circular, simple o doblemente enlazada.*

```
TNodo_Lista *elementos;
elementos=new TNodo_Lista;
if (elementos!=NULL)
{
  elementos->Siguiente=NULL;
  elementos->Anterior=NULL;
}
```

Ejemplo 6. *Creación de una lista con nodo cabecera.*

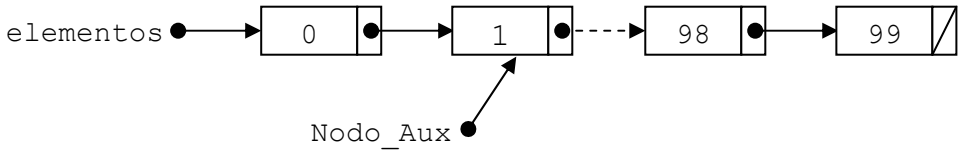
Comprobación si está vacía

```
return elementos==NULL;
// Para una lista simple o doblemente enlazada (circular o no),
// devolverá true si la lista no contiene nodos
```

```
return elementos->Siguiente==NULL;
// Para una lista con nodo cabecera, simple o doblemente enlazada
// devolverá true si la lista no contiene nodos
```

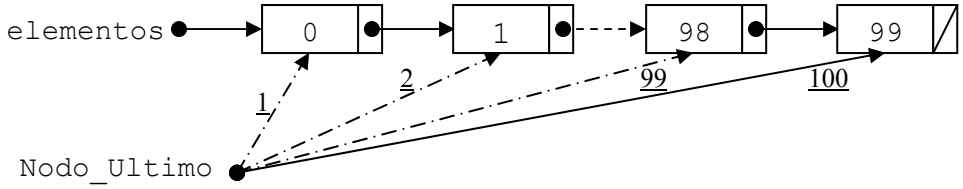
Ir al elemento siguiente

```
elementos->Datos=Valor; //Asigna un valor al primer elemento
                        //de la lista
elementos->Siguiente->Datos=Valor; //Asigna un valor al segundo
                                   //elemento de la lista.
```



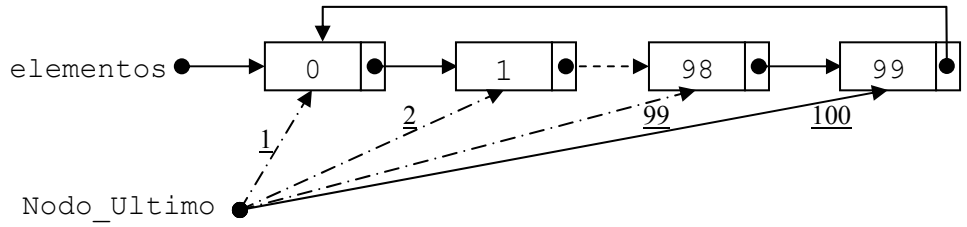

```
//También se podría haber hecho utilizando un puntero auxiliar:
TNodo_Lista *Nodo_Aux;
Nodo_Aux=elementos->Siguiente;
Nodo_Aux->Datos=Valor;   //Asigna un valor al segundo elemento.
```

Ir al último elemento



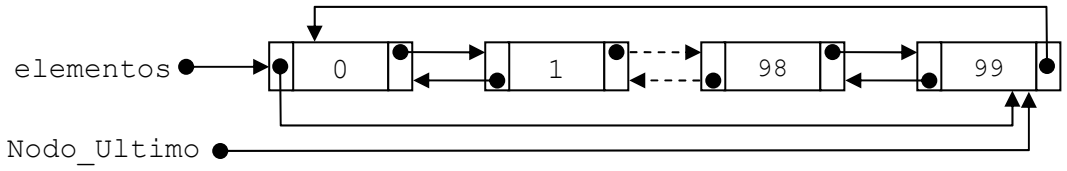

```
TNodo_Lista *Nodo_Ultimo=elementos;

//Si hay nodos busca el nodo que tenga NULL
if (elementos!=NULL)
  while (Nodo_Ultimo->Siguiente!=NULL)
    Nodo_Ultimo=Nodo_Ultimo->Siguiente; //Siguiente nodo
```

Ejemplo 7. *Ir al último elemento en una lista simple y doblemente enlazada.*




```
TNodo_Lista *Nodo_Ultimo=elementos; //Apunta al primer nodo

//Si hay nodos busca el nodo que tenga el puntero elementos
if (elementos!=NULL)
  while (Nodo_Ultimo->Siguiente!=elementos)
    Nodo_Ultimo=Nodo_Ultimo->Siguiente; //Siguiente nodo
```

Ejemplo 8. *Ir al último elemento en una lista circular simplemente enlazada*




```
TNodo_Lista *Nodo_Ultimo=elementos; //Apunta al primer nodo

//Si hay nodos accede directamente al último nodo
if (elementos!=NULL)
  Nodo_Ultimo=elementos->Anterior; //último nodo
```

Ejemplo 9. *Ir al último elemento en una lista circular doblemente enlazada.*

Recorrer una lista

Una operación frecuente es la de recorrer los nodos para realizar una búsqueda o actualización de datos.

Este recorrido añade condiciones de parada además de las condiciones normales para determinar el final de la lista.

```
TNodo_Lista *Nodo_Aux=elementos;
bool encontrado=false;

//condición de fin de lista y de parada
while (Nodo_Aux!=NULL && !encontrado)
  if (Nodo_Aux->Datos!=valor_a_buscar)
    Nodo_Aux=Nodo_Aux->Siguiente;
  else
    encontrado=true;
```

Ejemplo 10. *Ejemplo de búsqueda en una lista simple o doblemente enlazada.*

Se necesita una variable puntero auxiliar que vaya apuntando en cada momento al nodo que se quiera tratar.

Una vez tratado un nodo determinado se continúa con el nodo adyacente hasta que el puntero al nodo siguiente sea NULL o sea el primer nodo en caso de las listas circulares o bien se verifique una condición adicional de parada como puede observarse en el ejemplo 11.

En el ejemplo de la derecha vemos que utilizamos una variable v que cuenta el número de nodos recorridos.

Si alcanzamos el nodo solicitado o bien alcanzamos nuevamente el primer nodo el bucle termina.

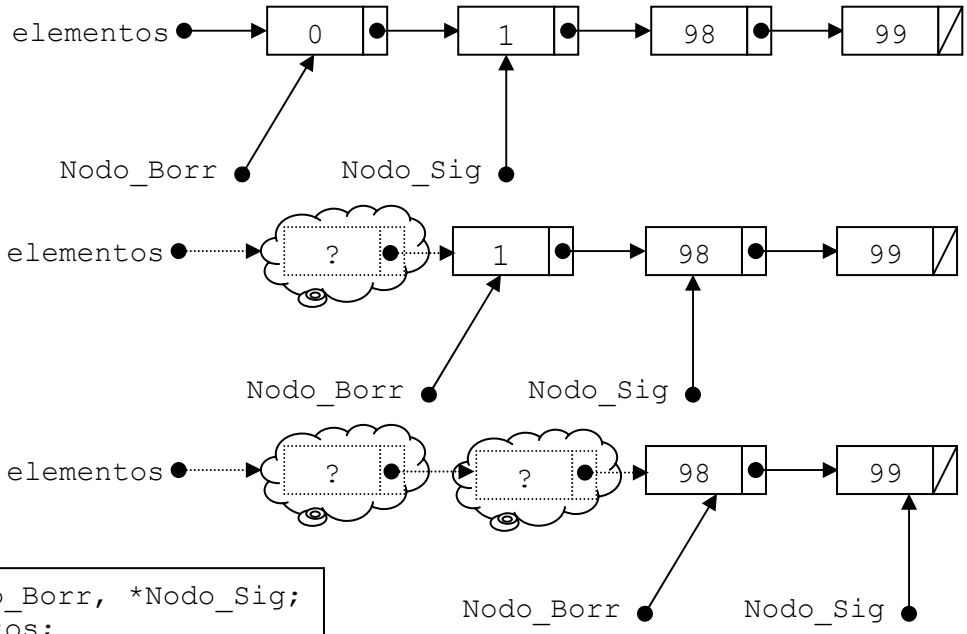
Finalmente hay que determinar si efectivamente se ha llegado al nodo correspondiente para su tratamiento.

```
TNodo_Lista *Nodo_Aux=elementos;
int v=1;

if (Nodo_Aux!=NULL) {
  //Condición de fin de lista y de parada
  while (Nodo_Aux->Siguiente!=elementos && v<i)
  {
    Nodo_Aux=Nodo_Aux->Siguiente;
    v++;
  }
  if (v==i)
    Nodo_Aux->Datos=x;
}
```

Ejemplo 11. *Ejemplo de búsqueda en una lista circular simple o doblemente enlazada, buscando el nodo que está situado en la posición que indica la variable i.*

Destrucción

Se ha de recorrer y eliminar cada uno de los nodos. Para ello se necesitan dos punteros temporales, uno que apunte al nodo a ser eliminado y otro que apunte al nodo siguiente para evitar que se pierda el acceso a los nodos posteriores.

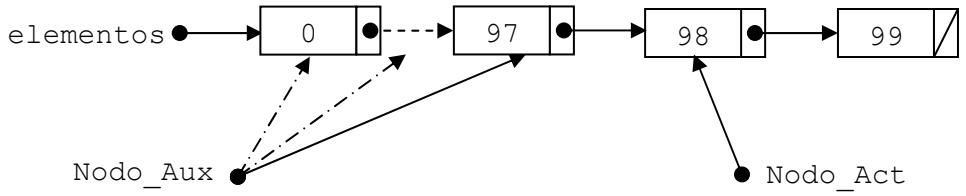



```
TNodo_Lista *Nodo_Borr, *Nodo_Sig;
Nodo_Borr=elementos;

while (Nodo_Borr!=NULL)
{
  Nodo_Sig=Nodo_Borr->Siguiente;
  delete Nodo_Borr;
  Nodo_Borr=Nodo_Sig;
}
elementos=NULL;
```

```
TNodo_Lista *Nodo_Borr, *Nodo_Sig;
Nodo_Borr=elementos;

while (Nodo_Borr->Siguiente!=elementos)
{
  Nodo_Sig=Nodo_Borr->Siguiente;
  delete Nodo_Borr;
  Nodo_Borr=Nodo_Sig;
}
delete Nodo_Borr;
elementos=NULL;
```

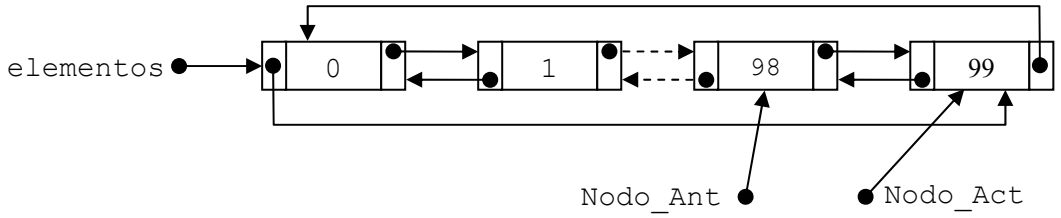
Ejemplo 12. *Destrucción de una lista simple o doblemente enlazada y en el código de la derecha la destrucción de una lista circular simple o doblemente enlazada.*

Ir al elemento anterior




```
TNodo_Lista *Nodo_Ant=NULL,      //Elemento anterior
            *Nodo_Act,           //Nodo Actual
            *Nodo_Aux=elementos;  //Primer nodo de la lista

if (Nodo_Aux!=NULL && Nodo_Act!=NULL && Nodo_Act!=elementos)
{
/* Mientras el nodo siguiente del nodo auxiliar no sea el
último y no sea el nodo actual, se pasa al siguiente
nodo */

  while (Nodo_Aux->Siguiente!=NULL &&
         Nodo_Aux->Siguiente!=Nodo_Act)
    Nodo_Aux=Nodo_Aux->Siguiente;
  if (Nodo_Aux->Siguiente==Nodo_Act)
    Nodo_Ant=Nodo_Aux;
  else
    Nodo_Ant=NULL;
}
```

Ejemplo 13. *Ir al elemento anterior en una lista simplemente enlazada. En el caso de que el nodo actual sea el primer nodo de la lista, el nodo anterior es NULL.*




```
TNodo_Lista *Nodo_Ant=NULL,   //Elemento anterior
            *Nodo_Act,        //Nodo Actual

if (Nodo_Act!=NULL)
  Nodo_Ant=Nodo_Act->Anterior;
```

Ejemplo 14. *Ir al elemento anterior en una lista doblemente enlazada. Si el nodo actual es el primer nodo de la lista, si esta es circular el nodo anterior es el último nodo de la lista, y si la lista no es circular el nodo anterior será NULL.*

Inserción de un nodo

El problema de la inserción de un nuevo nodo en una lista hay que estudiarlo desde las distintas situaciones que se pueden dar en el estado de la lista y desde las distintas posiciones en las que se desea insertar el nuevo nodo.

Estas situaciones son:

1. La lista esta vacía y la única posición a insertar es al principio de la lista.
2. La lista contiene nodos y las distintas posiciones a insertar son:
    a. Al principio de la lista.
    b. En cualquier posición intermedia distinta del principio.

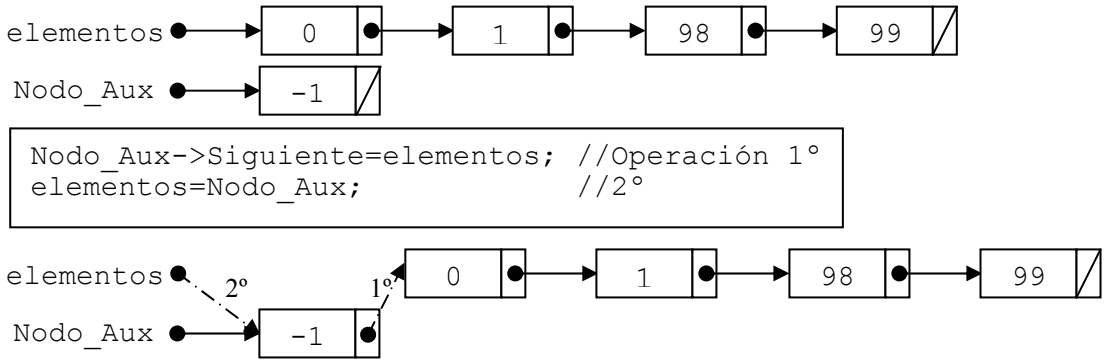
- Antes de todo proceso de inserción, hay un proceso de creación del nodo y actualización de la información que va a ser insertada en la lista.
- En el caso que la lista de datos no contenga ningún nodo, la inserción del nuevo nodo consiste en asignar el puntero del nodo auxiliar al puntero de la lista que deba contener la dirección del primer nodo.
- En el caso que la lista contenga nodos, el proceso de inserción del nuevo nodo dependerá del lugar donde se quiera insertar y del tipo de lista (simple o doblemente enlazada).

```
TNodo_Lista *Nodo_Aux=new TNodo_Lista;
if (Nodo_Aux!=NULL)
{
  Nodo_Aux->Datos=Valor_del_dato;
  Nodo_Aux->Siguiente=NULL;
  //Nodo_Aux->Anterior=NULL;
}
```

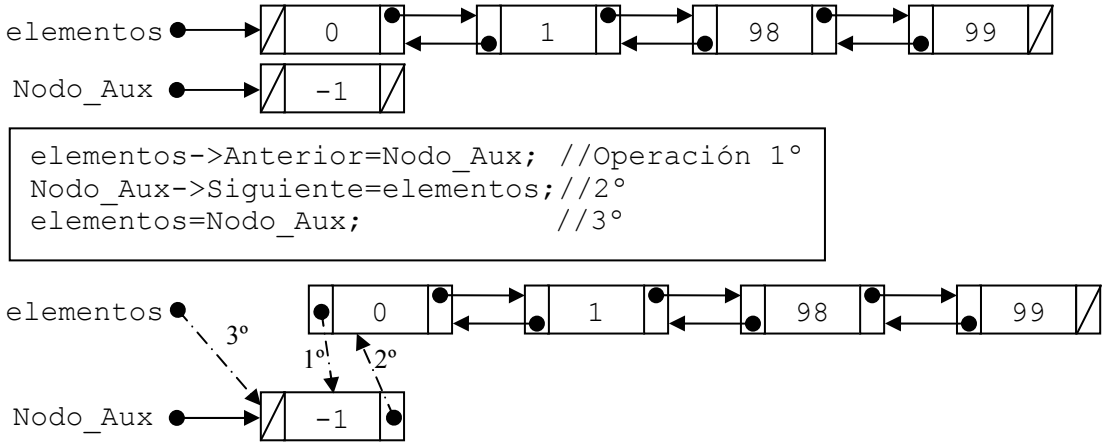
Ejemplo 15. *Ejemplo de creación de un nuevo nodo a insertar.*

```
elementos = Nodo_Aux;
```

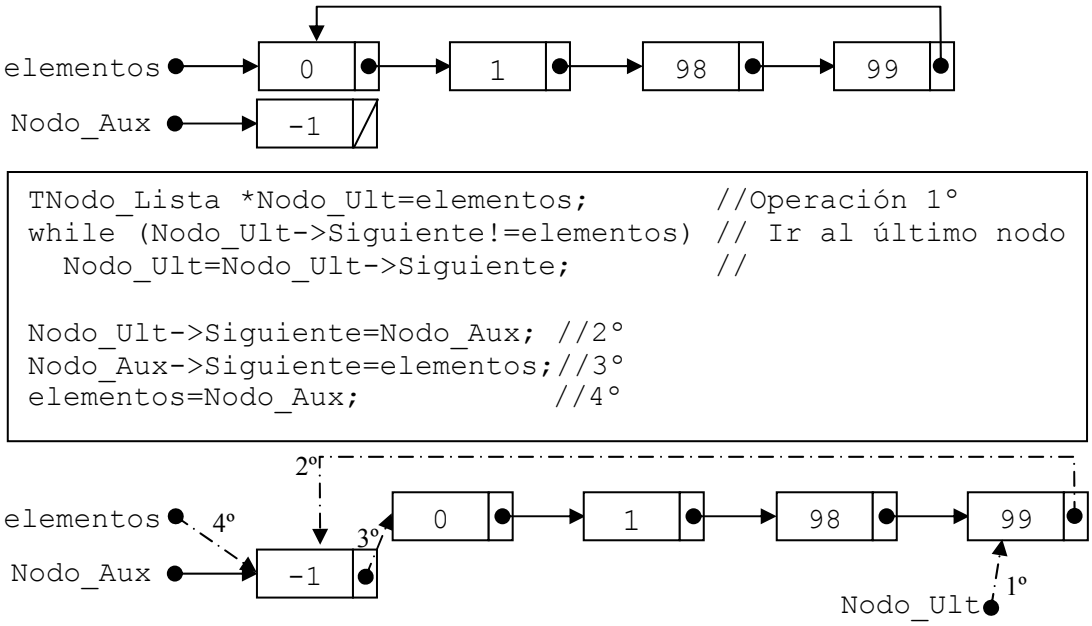
Al principio de la lista



Ejemplo 16. *Inserción al principio en una lista simplemente enlazada.*

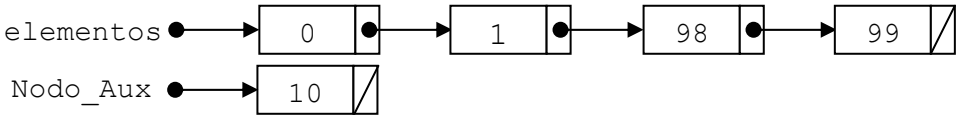


Ejemplo 17. *Inserción al principio en una lista doblemente enlazada.*

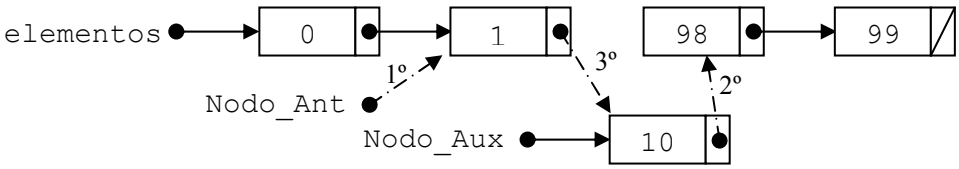


Ejemplo 18. *Inserción al principio en una lista circular simplemente enlazada.*

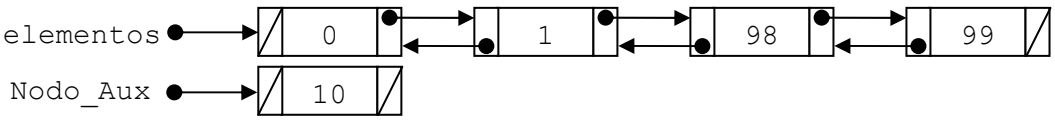
En cualquier posición de la lista



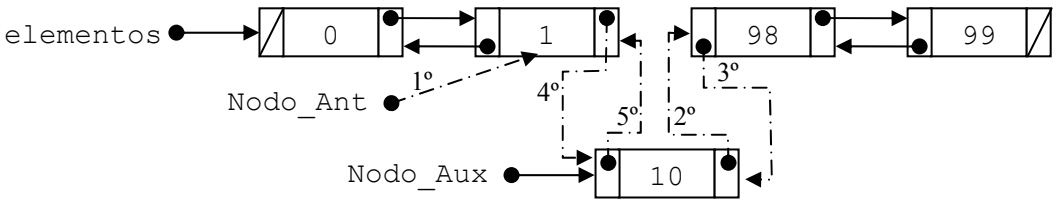

```
TNodo_Lista *Nodo_Ant=NULL,       //Elemento anterior
            *Nodo_Tmp=elementos; //Primer nodo de la lista

while (Nodo_Tmp!=NULL && Nodo_Tmp->Datos<Nodo_Aux->Datos)
{
  Nodo_Ant=Nodo_Tmp;              //Operación 1°
  Nodo_Tmp=Nodo_Tmp->Siguiente;
}
Nodo_Aux->Siguiente=Nodo_Ant->Siguiente; //2°
Nodo_Ant->Siguiente=Nodo_Aux;            //3ª
```



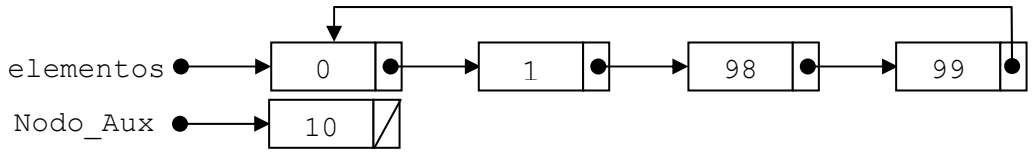
Ejemplo 19. *Inserción en cualquier posición de una lista simplemente enlazada.*



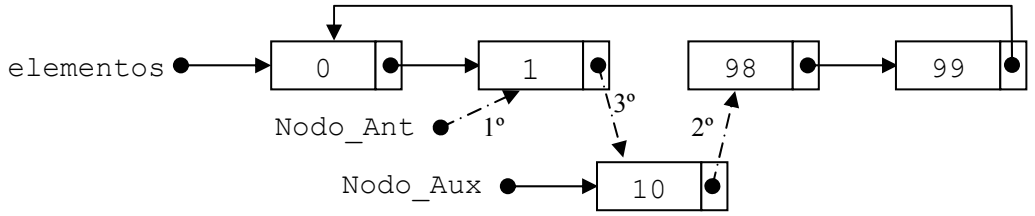

```
TNodo_Lista *Nodo_Ant=NULL,       //Elemento anterior
            *Nodo_Tmp=elementos; //Primer nodo de la lista

while (Nodo_Tmp!=NULL && Nodo_Tmp->Datos<Nodo_Aux->Datos)
{
  Nodo_Ant=Nodo_Tmp;              //Operación 1°
  Nodo_Tmp=Nodo_Tmp->Siguiente;
}
Nodo_Aux->Siguiente=Nodo_Ant->Siguiente; //2°
if (Nodo_Tmp!=NULL)
  Nodo_Ant->Siguiente->Anterior=Nodo_Aux;  //3°
Nodo_Ant->Siguiente=Nodo_Aux;            //4°
Nodo_Aux->Anterior=Nodo_Ant;             //5°
```



Ejemplo 20. *Inserción en cualquier posición de una lista doblemente enlazada.*



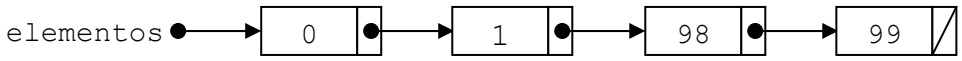

```
TNodo_Lista *Nodo_Ant=elementos,      //Elemento anterior
            *Nodo_Tmp=elementos->Siguiente; //Primer nodo de la lista

while (Nodo_Tmp!=elementos && Nodo_Tmp->Datos<Nodo_Aux->Datos)
{
  Nodo_Ant=Nodo_Tmp;              //Operación 1°
  Nodo_Tmp=Nodo_Tmp->Siguiente;
}
Nodo_Aux->Siguiente=Nodo_Ant->Siguiente; //2°
Nodo_Ant->Siguiente=Nodo_Aux;            //3°
```



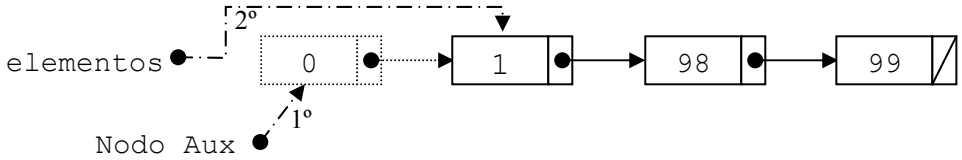
Ejemplo 21. *Inserción en cualquier posición de una lista circular simplemente enlazada.*

Eliminación de un nodo

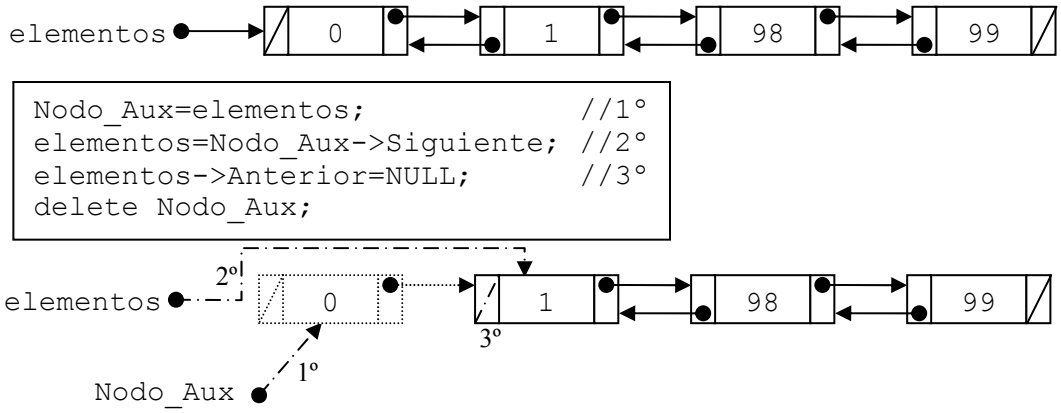
Al principio de la lista



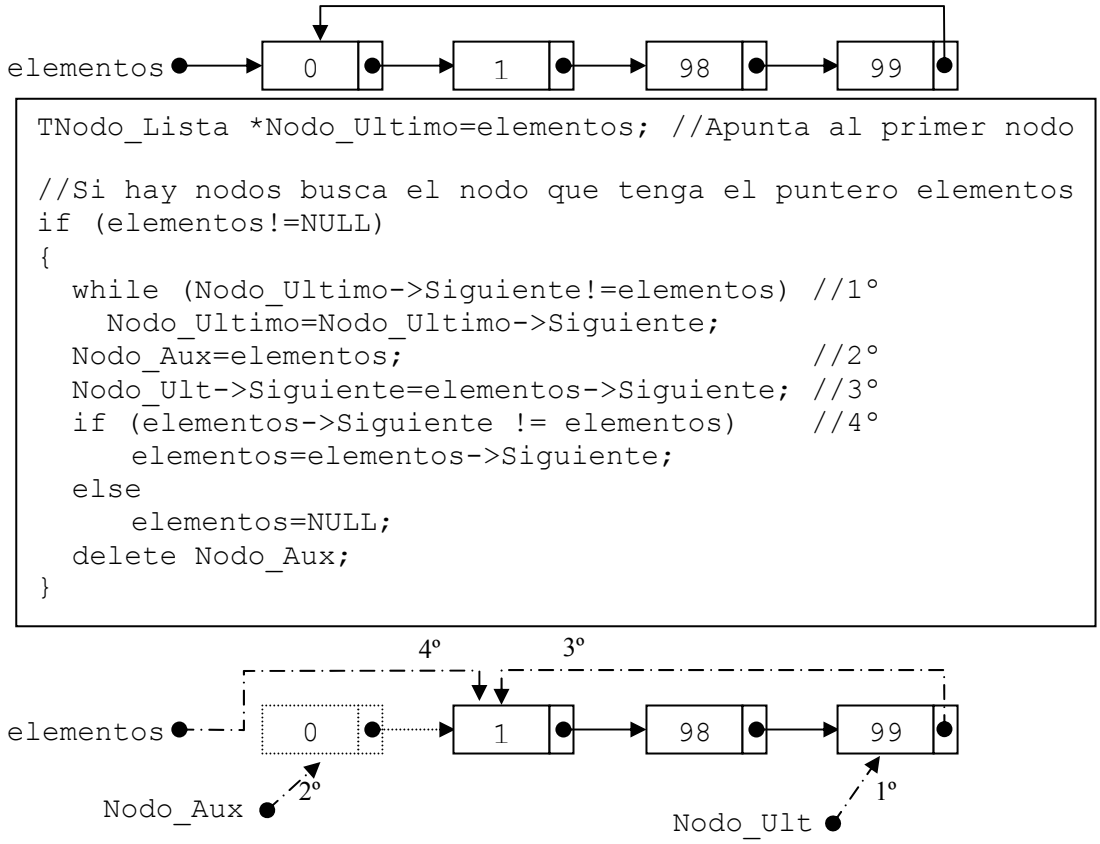

```
Nodo_Aux=elementos;            //1°
elementos=Nodo_Aux->Siguiente; //2°
delete Nodo_Aux;
```



Ejemplo 22. *Eliminación al principio de una lista simplemente enlazada.*

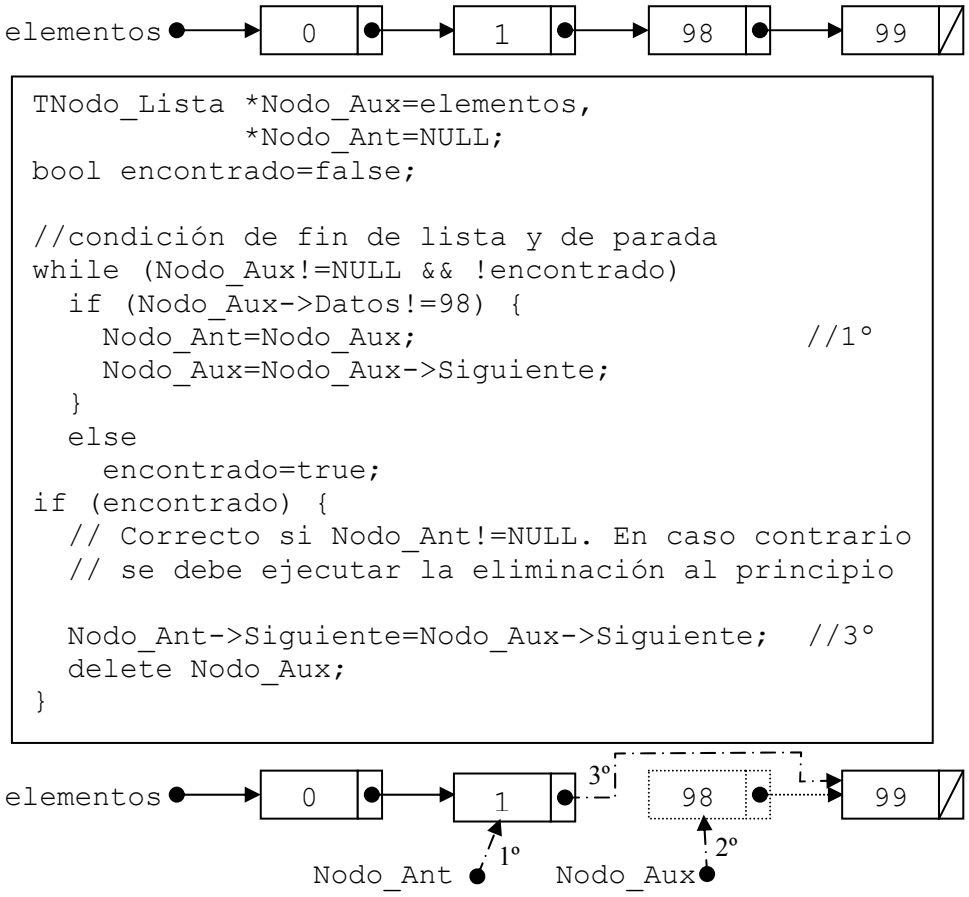


Ejemplo 23. *Eliminación al principio de una lista doblemente enlazada.*



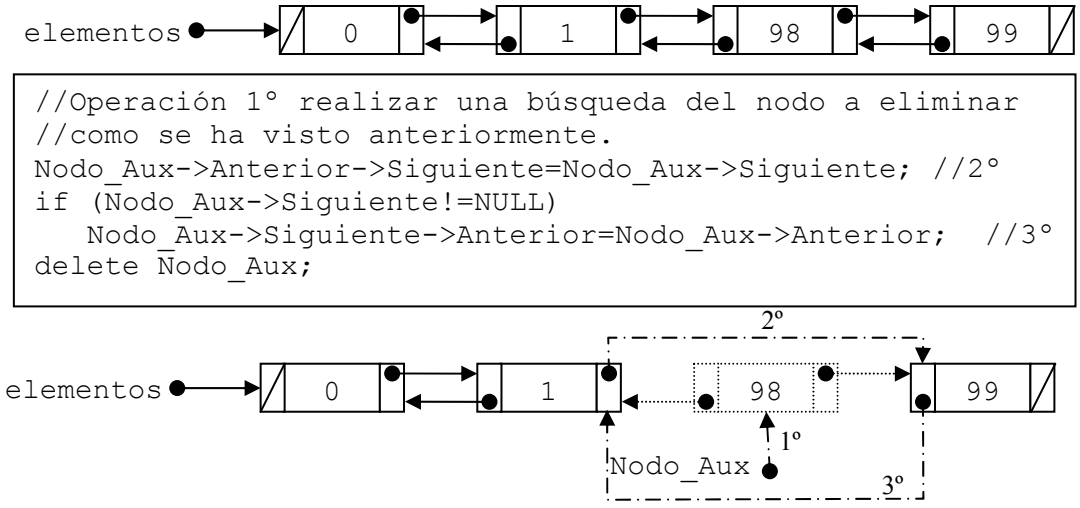
Ejemplo 24. *Eliminación al principio de una lista circular simplemente enlazada.*

En cualquier posición de la lista



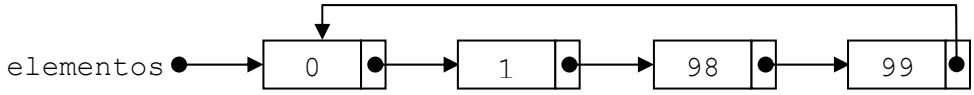

```
TNodo_Lista *Nodo_Aux=elementos,
            *Nodo_Ant=NULL;
bool encontrado=false;

//condición de fin de lista y de parada
while (Nodo_Aux!=NULL && !encontrado)
  if (Nodo_Aux->Datos!=98) {
    Nodo_Ant=Nodo_Aux;                        //1°
    Nodo_Aux=Nodo_Aux->Siguiente;
  }
  else
    encontrado=true;
if (encontrado) {
  // Correcto si Nodo_Ant!=NULL. En caso contrario
  // se debe ejecutar la eliminación al principio

  Nodo_Ant->Siguiente=Nodo_Aux->Siguiente;  //3°
  delete Nodo_Aux;
}
```

Ejemplo 25. *Eliminación en cualquier posición de una lista simplemente enlazada.*

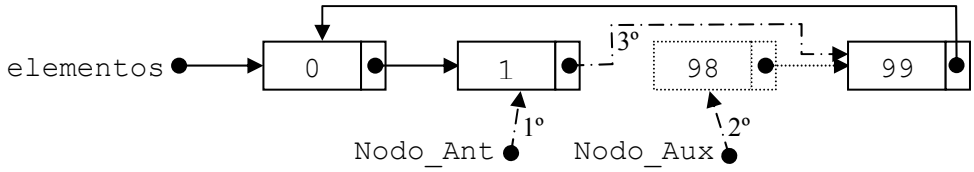



```
//Operación 1° realizar una búsqueda del nodo a eliminar
//como se ha visto anteriormente.
Nodo_Aux->Anterior->Siguiente=Nodo_Aux->Siguiente; //2°
if (Nodo_Aux->Siguiente!=NULL)
   Nodo_Aux->Siguiente->Anterior=Nodo_Aux->Anterior;  //3°
delete Nodo_Aux;
```

Ejemplo 26. *Eliminación en cualquier posición de una lista doblemente enlazada.*




```
TNodo_Lista *Nodo_Aux=elementos,
            *Nodo_Ant=NULL;
bool encontrado=false;

//condición de fin de lista y de parada
while (Nodo_Aux->Siguiente!=elementos && !encontrado)
  if (Nodo_Aux->Datos!=98)
  {
    NodoAnt=Nodo_Aux;                        //1°
    Nodo_Aux=Nodo_Aux->Siguiente;
  }
  else
    encontrado=true;
if (encontrado || Nodo_Aux->Datos==98)
{
  //Correcto si Nodo_Ant!=NULL. En caso contrario
  //se debe ejecutar la eliminación al principio

  Nodo_Ant->Siguiente=Nodo_Aux->Siguiente;  //3°
  delete Nodo_Aux;
}
```



Ejemplo 27. *Eliminación en cualquier lugar de una lista circular simplemente enlazada.*

1.2.2 Implementación del TAD Lista con nodos enlazados.

La implementación es mediante una lista simplemente enlazada utilizando el lenguaje C++ donde el tipo de los elementos es real (**float**), haciendo uso de diseño modular y orientación a objetos.

```
// FICHERO TADLista.h

#include <iostream>
using namespace std;

struct TNodo_Lista
{
  float Datos;            // Dato a almacenar en cada nodo
  TNodo_Lista *Siguiente; // Puntero al siguiente nodo
};


class lista
{
     TNodo_Lista *elementos;   // Puntero al primer nodo de la lista.
     int  n;                   // n° de nodos que tiene la lista

     // Devuelve un puntero al elemento anterior al elemento i
     // Devuelve NULL si es el primero
     TNodo_Lista *Anterior(int i);

  public:
      lista();  // constructor de la clase
     ~lista();  // destructor de la clase
     lista(float e);
     bool esvacia();
     int longitud();
     void anadirIzq(float e);
     void anadirDch(float e);
     void eliminarIzq();
     void eliminarDch();
     float observarIzq();
     float observarDch();
     void concatenar(lista l);
     bool pertenece(float e);
     void insertar(int i, float e);
     void eliminar(int i);
     void modificar(int i, float e);
     float observar(int i);
     int posicion(float e);
};
```

```
//FICHERO TADLista.cpp
#include "TADLista.h"

lista::lista()
{
  elementos=NULL;
  n=0;
}

lista::~lista()
{
  TNodo_Lista *Nodo_Borr=elementos,*Nodo_Sig;

  while (Nodo_Borr!=NULL) {
    Nodo_Sig=Nodo_Borr->Siguiente;
    delete Nodo_Borr;
    Nodo_Borr=Nodo_Sig;
  }
  elementos=NULL;
  n=0;
}

TNodo_Lista *lista::Anterior(int i)
{
  TNodo_Lista *Nodo_Aux=elementos,*Nodo_Ant=NULL;
  int v=1;
  if (Nodo_Aux!=NULL)
    while (Nodo_Aux!=NULL && v<i)  {
      Nodo_Ant=Nodo_Aux;
      Nodo_Aux=Nodo_Aux->Siguiente;
      v++;
    }
  return Nodo_Ant;
};

void lista::insertar(int i, float e)
{
  TNodo_Lista *Nodo_Aux=new TNodo_Lista, *Nodo_Ant;
  if (Nodo_Aux!=NULL) {
    Nodo_Aux->Datos=e;
    Nodo_Aux->Siguiente=NULL;
    Nodo_Ant= Anterior(i);
    if (Nodo_Ant==NULL)  {
      Nodo_Aux->Siguiente=elementos;
      elementos=Nodo_Aux;
    }
    else {
```

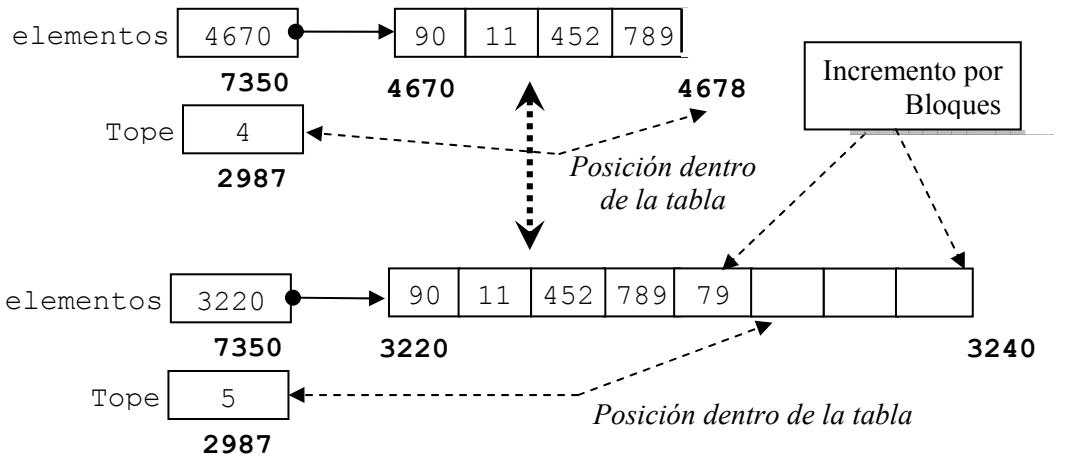
```
      Nodo_Aux->Siguiente=Nodo_Ant->Siguiente;
      Nodo_Ant->Siguiente=Nodo_Aux;
    }
    n++;
  }
}

void lista::eliminar(int i)
{
  TNodo_Lista *Nodo_Ant,*Nodo_Aux;
  Nodo_Ant=Anterior(i);

  if (Nodo_Ant==NULL)  {
    Nodo_Aux=elementos;
    elementos=Nodo_Aux->Siguiente;
  }
  else  {
    Nodo_Aux=Nodo_Ant->Siguiente;
    Nodo_Ant->Siguiente=Nodo_Aux->Siguiente;
  }
  delete Nodo_Aux;
  n--;
}

void lista::modificar(int i, float e)
{
  TNodo_Lista *Nodo_Ant=Anterior(i),*Nodo_Aux;
  if (Nodo_Ant==NULL)
    Nodo_Aux=elementos;
  else
    Nodo_Aux=Nodo_Ant->Siguiente;
  Nodo_Aux->Datos=e;
}

float lista::observar(int i)
{
  TNodo_Lista *Nodo_Ant=Anterior(i),*Nodo_Aux;
  if (Nodo_Ant==NULL)
    Nodo_Aux=elementos;
  else
    Nodo_Aux=Nodo_Ant->Siguiente;
  return Nodo_Aux->Datos;
}
```

```
bool lista::esvacia ()
{
  return (n == 0);
}

int lista::longitud ()
{
  return n;
}

int lista::posicion(float e)
{
  TNodo_Lista *Nodo_Aux=elementos;
  bool encontrado=false;
  int v=1;
  while (Nodo_Aux!=NULL && !encontrado)
    if (Nodo_Aux->Datos!=e)
    {
      Nodo_Aux=Nodo_Aux->Siguiente;
      v++;
    }
    else
      encontrado=true;

  return (encontrado?v:-1);
}

lista::lista(float e)
{
  elementos=new TNodo_Lista;
  elementos->Datos=e;
  elementos->Siguiente=NULL;
  n=1;
}
```

2. Implementación dinámica del TAD Pila.

Para la implementación dinámica del TAD Pila vamos a utilizar la misma especificación que se puede consultar en el tema anterior.

2.1 Con tablas dinámicas.



Ejemplo 28. *Diseño del TAD Pila mediante tablas dinámicas.*

La representación del TAD Pila que implementaremos es aquella que utiliza una variable entera denominada *Tope* para almacenar el índice de la tabla donde está situado el tope de la pila. Esta variable aumenta hacia valores altos se realiza la operación de apilar y disminuye cuando se realiza la operación de desapilar.

Implementación.

Implementamos en C++ una pila donde el tipo de los elementos son cadenas de caracteres (clase *Cadena*), haciendo uso de diseño modular y orientación a objetos.

```
// FICHERO TADPila.h
#include <iostream>
#include "TADCadena.h"
#define INCREMENTO 4
using namespace std;

class pila
{
   Cadena* elementos;  // elementos de la pila
   int Tope;           // tope de la pila
   int Tama;           // tamaño máximo de la tabla
 public:
```

```
   pila();             // constructor de la clase
   ~pila();            // destructor de la clase
   void apilar(Cadena e);
   void desapilar();
   bool esvacia();
   Cadena cima() ;
   int longitud();
};
```

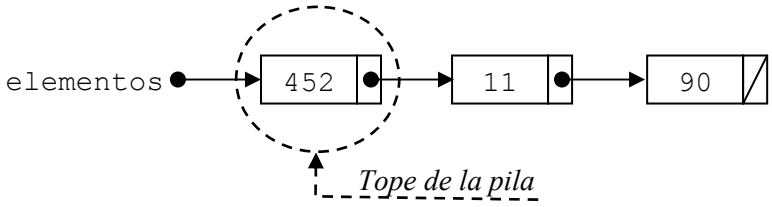
```
//FICHERO TADPila.cpp
#include "TADPila.h"

pila::pila()
{
  elementos=new Cadena[INCREMENTO];
  Tama=(elementos!=NULL ? INCREMENTO : -1);
  Tope=-1;
}

pila::~pila()
{
  if (elementos!=NULL)
    delete [] elementos;
  elementos=NULL;
  Tope=-1;
  Tama=0;
}

void pila::apilar(Cadena e)
{
  if (Tope+1==Tama)
  {
    Cadena *NuevaZona=new Cadena[Tama+INCREMENTO];
    if (NuevaZona!=NULL) {
      for (int i=0;i<Tama; i++)
        NuevaZona[i]=elementos[i];
      Tama+=INCREMENTO;
      delete [] elementos;
      elementos=NuevaZona;
    }
  };

  if (Tope+1<Tama)
    elementos[++Tope]=e;
}
```

```
void pila::desapilar()
{
  Tope--;
  if (Tama-Tope>INCREMENTO && Tama>INCREMENTO)
  {
    Cadena *NuevaZona=new Cadena[Tama-INCREMENTO];
    if (NuevaZona!=NULL)
    {
      Tama-=INCREMENTO;
      for (int i=0;i<Tama; i++)
        NuevaZona[i]=elementos[i];
      delete [] elementos;
      elementos=NuevaZona;
    };
  };
}

Cadena pila::cima()
{
  return elementos[Tope];
}

bool pila::esvacia()
{
  return (Tope == -1);
}

int pila::longitud()
{
  return (Tope+1);
}
```

2.2 Con nodos enlazados.

La implementación dinámica de pilas utilizando nodos enlazados utiliza el primer nodo como elemento tope de la lista.

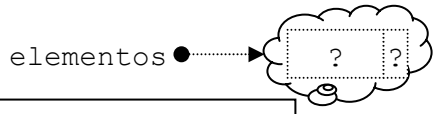
Este tipo de estructura es más simple y eficiente además de utilizar menos recursos de memoria.



Ejemplo 29. *Diseño de una Pila dinámica utilizando el primer nodo de la lista como el tope de la Pila.*

2.2.1 Operaciones básicas sobre pilas

Declaración




```
struct TNodo_Pila
{
  Tipo_elemento  Datos;  //Datos a almacenar en cada nodo
  TNodo_Pila *Siguiente; //Puntero al siguiente nodo
};

TNodo_Pila *elementos; //Puntero al primer elemento
```

Ejemplo 30. *Declaración de una Pila mediante una lista simplemente enlazada.*

Creación

```
//La Pila no contiene ningún nodo.
TNodo_Pila *elementos=NULL;
```

Ejemplo 31. *Creación de una Pila dinámica vacía.*

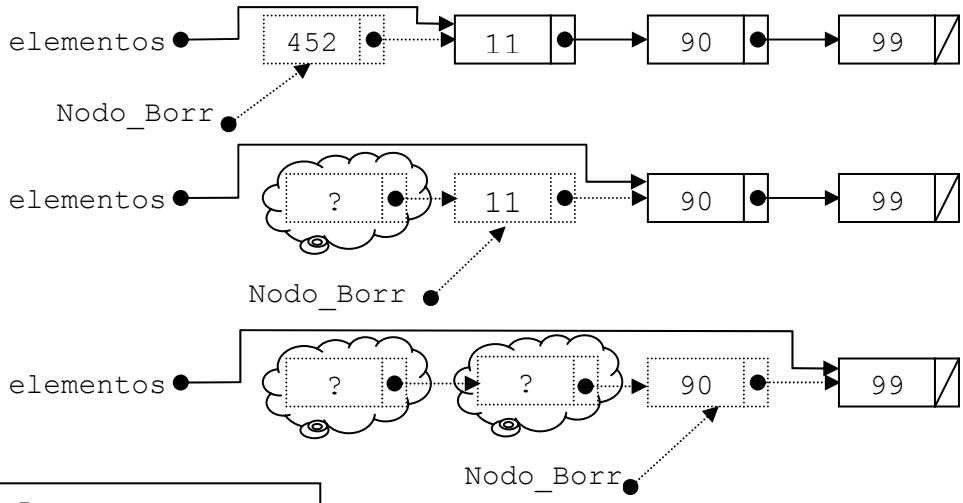
```
TNodo_Pila *elementos;
elementos=new TNodo_Pila;
if (elementos!=NULL)
  elementos->Siguiente=NULL;
```

Ejemplo 32. *Creación de una Pila con nodo cabecera.*

Comprobación si está vacía

```
return elementos==NULL; //Devolverá true si la Pila está vacía
```

Destrucción



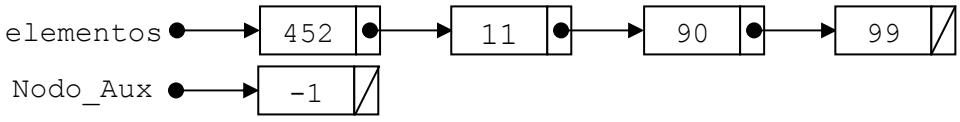

```
TNodo_Pila *Nodo_Borr;

while (elementos!=NULL)
{
  Nodo_Borr=elementos;
  elementos=Nodo_Borr->Siguiente;
  delete Nodo_Borr;
}
```

Ejemplo 33. *La eliminación de una Pila dinámica es idéntica a la eliminación de una lista dinámica simplemente enlazada.*

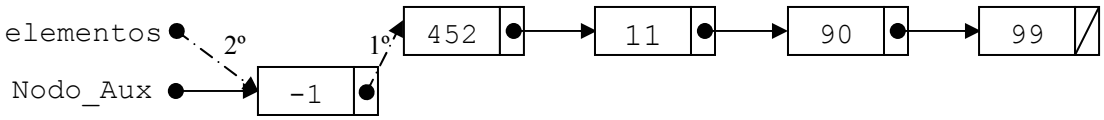
Tope de una pıla

```
if (elementos!=NULL)
  return elementos->Datos;
```

Apilar elementos

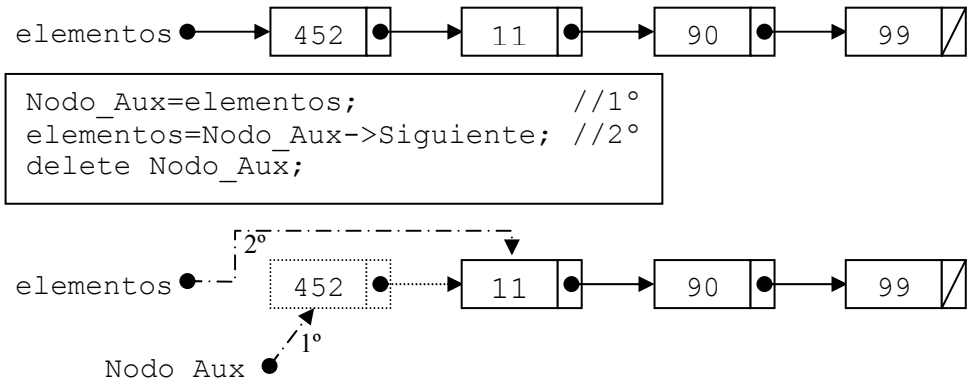



```
Nodo_Aux->Siguiente=elementos; //Operación 1°
elementos=Nodo_Aux;            //2°
```



Ejemplo 34. *El proceso de apilado se realiza siempre por el primer nodo de la lista.*

Desapilar elementos



Ejemplo 35. *El proceso de desapilar se realiza por el primer nodo de la lista dinámica.*

2.2.2 Implementación del TADPila con nodos enlazados.

La implementación del TADPila ha sido realizada utilizando una lista simplemente enlazada con el lenguaje C++ donde el tipo de los elementos es de la clase Cadena, haciendo uso de diseño modular y orientación a objetos.

```
// FICHERO TADPila.h
#include <iostream>
#include "TADCadena.h"
using namespace std;

struct TNodo_Pila
{
   Cadena Datos;           // Dato a almacenar en cada nodo
   TNodo_Pila *Siguiente;  // Puntero al siguiente nodo
};

class pila
{
   TNodo_Pila *elementos; // elementos de la pila
   int  n;                // n° de nodos que tiene la lista
 public:
   pila();                / /constructor de la clase
   ~pila();               // destructor de la clase
   void apilar(Cadena e);
   void desapilar();
   bool esvacia();
   Cadena cima() ;
   int longitud();
};
```

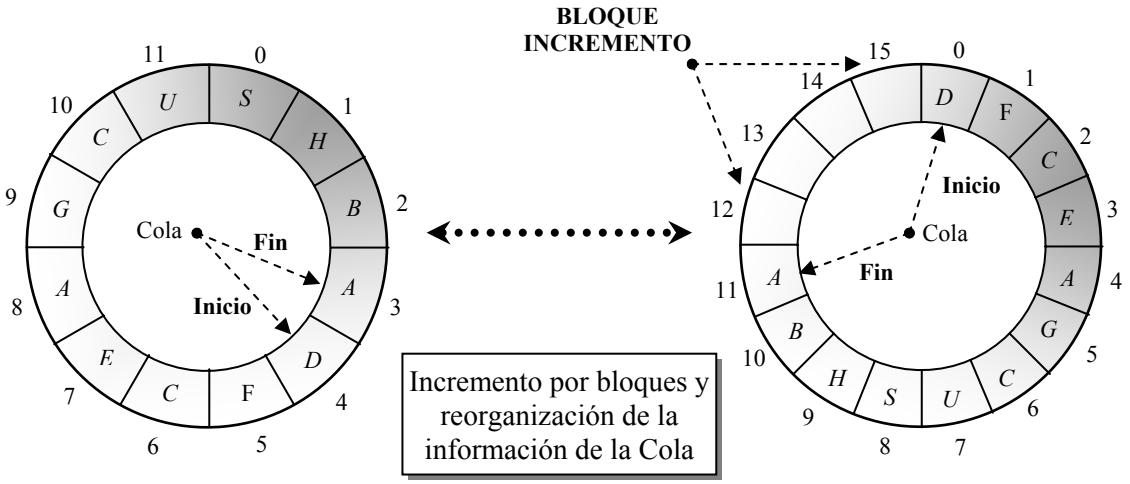
```
//FICHERO TADPila.cpp
#include "TADPila.h"

pila::pila() {
  elementos=NULL;
  n=0;
}

pila::~pila() {
  TNodo_Pila *Nodo_Borr;
  while (elementos!=NULL)  {
    Nodo_Borr=elementos;
    elementos=Nodo_Borr->Siguiente;
    delete Nodo_Borr;
  }
  n=0;
}

void pila::apilar(Cadena e) {
  TNodo_Pila *Nodo_Aux=new TNodo_Pila;
  if (Nodo_Aux!=NULL) {
    Nodo_Aux->Datos=e;
    Nodo_Aux->Siguiente=elementos;
    elementos=Nodo_Aux;
    n++;
  }
}

void pila::desapilar() {
  TNodo_Pila *Nodo_Aux=elementos;

  elementos=Nodo_Aux->Siguiente;
  delete Nodo_Aux;
  n--;
}

Cadena pila::cima() {
  return elementos->Datos;
}

bool pila::esvacia() {
  return elementos==NULL;
}

int pila::longitud() {
  return n;
}
```

3. Implementación dinámica del TAD Cola.

Para la implementación dinámica del TAD Cola vamos a utilizar la misma especificación del tema anterior.

3.1 Con tablas dinámicas.

- Se va a utilizar la representación de colas circulares mediante tablas dinámicas, donde tanto principio (`inicio`) como el final (`fin`) de la cola son variables, es decir, estos índices recorrerán los posiciones de la tabla de manera cíclica realizando la operación de encolar por el final de la cola (`fin`), y la de desencolar por la cabecera de la cola (`inicio`).
- Cada vez que se modifica dinámicamente el tamaño de la cola, los elementos de la nueva cola se reorganizaran de manera que el inicio de la cola volverá a la posición inicial de la tabla (índice 0) y el final de la cola se adecuará al nuevo tamaño de la tabla.




```
// FICHERO TADCola.h
#include <iostream>
#include "TADPersona.h"
#define INCREMENTO 4
using namespace std;

class cola
{
   Persona *elementos; //elementos de la cola
   int inicio, fin;    //principio y fin de la cola
   int Tama;           //Capacidad de la tabla
   int ne;             //N° de elementos
 public:
   cola();             // constructor de la clase
   ~cola();            // destructor de la clase
```

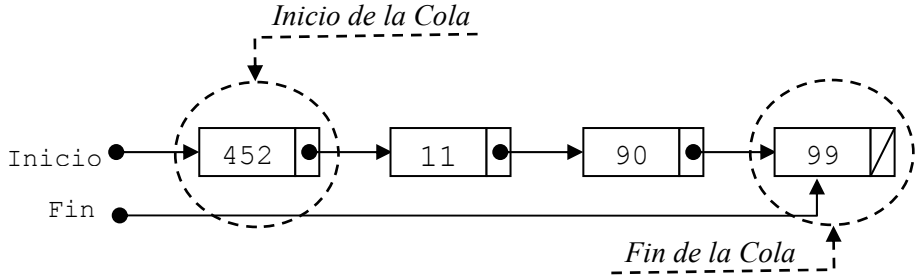
```
   void encolar(Persona e);
   void desencolar();
   bool esvacia();
   Persona primero() ;
   int longitud();
};
```

```
//FICHERO TADCola.cpp
#include "TadCola.h"

cola::cola()
{
  elementos=new Persona[INCREMENTO];
  if (elementos!=NULL)   {
    ne=fin=inicio=0;
    Tama=INCREMENTO;
  }
  else   {
    ne=fin=inicio=-1;
    Tama=-1;
  }
}
cola::~cola()
{
  if (elementos!=NULL)
    delete [] elementos;
  elementos=NULL;
  ne=fin=inicio=-1;
  Tama=0;
}

Persona cola::primero()
{
  return elementos[inicio];
}

bool cola::esvacia()
{
  return (ne==0);
}

int cola::longitud()
{
  return ne;
}
```

```
void cola::encolar(Persona e)
{
  if (ne==Tama) {
    Persona *NuevaZona=new Persona[Tama+INCREMENTO];
    if (NuevaZona!=NULL)  {
      for (int i=0;i<ne; i++) {
          NuevaZona[i]=elementos[inicio];
          inicio++;
          if (inicio==Tama) // inicio=(inicio+1)%Tama
              inicio=0;
      }
      inicio=0;
      fin=ne;
      Tama+=INCREMENTO;
      delete [] elementos;
      elementos=NuevaZona;
    }
  };
  if (ne<Tama) {
    elementos[fin]=e;
    fin=(fin+1)%Tama;
    ne++;
  }
}
void cola::desencolar()
{
  inicio++;            //  inicio=(inicio+1)%Tama;
  if (inicio==Tama)
    inicio=0;
  ne--;

  if (Tama-ne>=INCREMENTO && Tama>INCREMENTO)   {
    Persona *NuevaZona=new Persona[Tama-INCREMENTO];
    if (NuevaZona!=NULL)     {
      for (int i=0;i<ne; i++)  {
        NuevaZona[i]=elementos[inicio++];
        if (inicio==Tama)
          inicio=0;
      }
      Tama-=INCREMENTO;
      inicio=0;
      fin=Tama;
      delete [] elementos;
      elementos=NuevaZona;
    };
  };
}
```

3.2 Con nodos enlazados.

Se utilizan dos punteros, uno que apunta al primer nodo de la estructura para que haga el papel de cabecera (`inicio`) y otro que siempre apunta al último nodo para que haga el papel de final de la cola (`fin`).

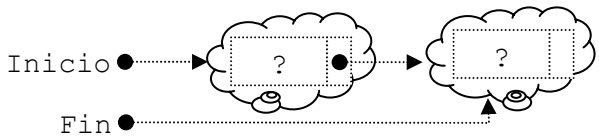


Ejemplo 36. *Diseño de una Cola dinámica utilizando dos punteros, uno al inicio de la Cola (primer nodo de la lista) y otro puntero al final de la Cola (último nodo de la lista).*

La estructura interna más simple, eficiente y que utiliza menos recursos de memoria es la lista simplemente enlazada, teniendo en cuenta que la operación de encolar se realiza al final de la lista y la operación de desencolar se realiza por el inicio de la lista.

3.2.1 Operaciones básicas sobre colas.

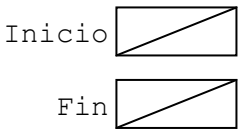
Declaración




```
struct TNodo_Cola
{
  Tipo_elemento  Datos;    //Datos a almacenar en cada nodo
  TNodo_Cola *Siguiente;   //Puntero al siguiente nodo
};
TNodo_Cola *Inicio;     //Puntero al inicio de la cola.
TNodo_Cola *Fin;        //Puntero al fin de la cola.
```

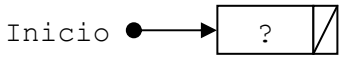
Ejemplo 37. *Declaración de una cola dinámica mediante dos variables puntero, uno al inicio y otro al fin de la cola.*

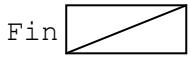
Creación




```
//La cola no contiene ningún nodo.
Inicio=Fin=NULL;
```

Ejemplo 38. *Creación de una Cola dinámica vacía.*






```
TNodo_Cola *Inicio,*Fin;
Inicio=new TNodo_Cola;
if (Inicio!=NULL)
  Inicio->Siguiente=NULL;
Fin=NULL;
```

Ejemplo 39. *Creación de una Cola con nodo cabecera.*

Comprobación si está vacía

```
return Inicio==NULL;  //Devolveré true si la Cola está vacía.
```

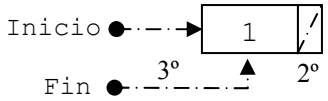
Destrucción

Para realizar una destrucción de una cola se ha de recorrer y eliminar cada uno de los nodos de manera que no se pierda información durante dicho proceso.

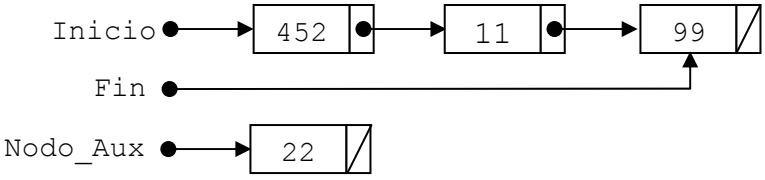
```
TNodo_Cola *Nodo_Borr;
while (Inicio!=NULL)
{
  Nodo_Borr=Inicio;
  Inicio=Nodo_Borr->Siguiente;
  delete Nodo_Borr;
}
```

Encolar

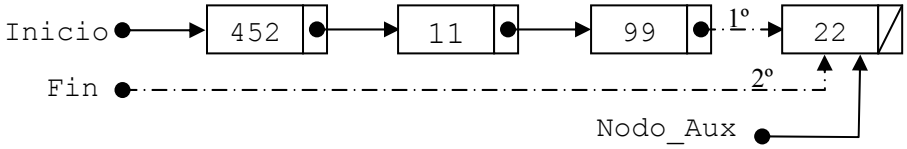
```
TNodo_Cola *Inicio=new TNodo_Cola; //Operación 1°
if (Inicio!=NULL) {
  Inicio->Datos=1;
  Inicio->Siguiente=NULL;         // Operación 2°
}
Fin=Inicio;                       //Operación 3°
```



Ejemplo 40. *Proceso de encolar en una Cola dinámica vacía.*

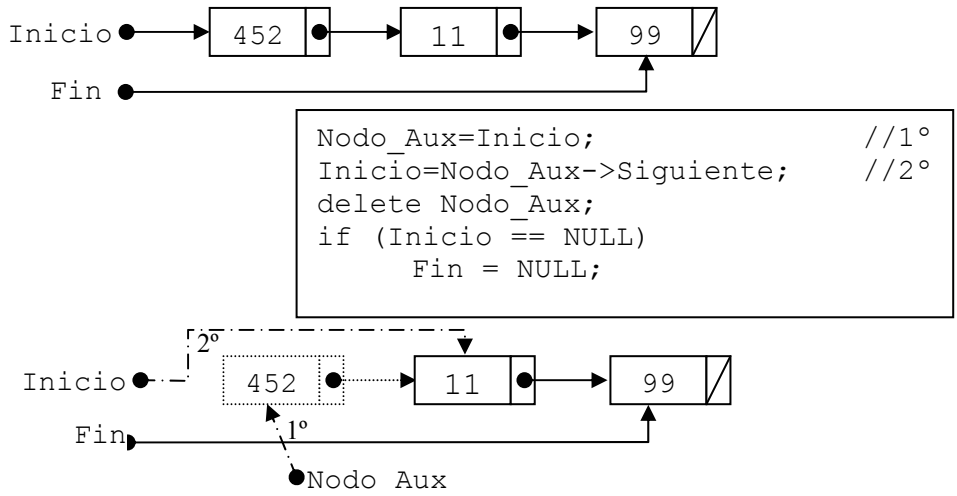



```
Fin->Siguiente=Nodo_Aux; //Operación 1°
Fin=Nodo_Aux;            //Operación 2°
```



Ejemplo 41. *Proceso de encolar en una Cola dinámica no vacía.*

Desencolar



Ejemplo 42. *Proceso de desencolar de una Cola dinámica.*

3.2.2 Implementación del TAD Cola con nodos enlazados.

La implementación del TAD Cola ha sido realizada utilizando una lista simplemente enlazada utilizando el lenguaje C++ donde el tipo de los elementos es de la clase Cadena, haciendo uso de diseño modular y orientación a objetos.

```
// FICHERO TADCola.h
#include <iostream>
#include "TADCadena.h"
using namespace std;

struct TNodo_Cola   {
   Cadena Datos;            //Datos a almacenar en cada nodo
   TNodo_Cola *Siguiente;   //Puntero al siguiente nodo
};

class cola {
   TNodo_Cola *Inicio;    //Puntero al inicio de la cola.
   TNodo_Cola *Fin;       //Puntero al fin de la cola.
   int  ne;               //n° de nodos que tiene la lista
 public:
   cola();                //constructor de la clase
   ~cola();               //destructor de la clase
   void encolar(Cadena e);
   void desencolar();
   bool esvacia();
   Cadena primero() ;
   int longitud();
```

```
};
//FICHERO TADCola.cpp
#include "TADCola.h"

cola::cola()
{
  Inicio=NULL;
  Fin=NULL;
  ne=0;
}

cola::~cola()
{
  TNodo_Cola *Nodo_Borr;
  while (Inicio!=NULL)
  {
    Nodo_Borr=Inicio;
    Inicio=Nodo_Borr->Siguiente;
    delete Nodo_Borr;
  }
  Fin=NULL;
  ne=0;
}

void cola::encolar(Cadena e) {
  TNodo_Cola *Nodo_Aux=new TNodo_Cola;
  if (Nodo_Aux!=NULL)
  {
    Nodo_Aux->Datos=e;
    Nodo_Aux->Siguiente=NULL;
    if (Inicio!=NULL)
      Fin->Siguiente=Nodo_Aux;
    else
      Inicio=Nodo_Aux;
    Fin=Nodo_Aux;
    ne++;
  }
}

void cola::desencolar() {
  TNodo_Cola *Nodo_Aux=Inicio;
  Inicio=Nodo_Aux->Siguiente;
  delete Nodo_Aux;
  ne--;
  if (ne == 0)
    Fin = NULL;
}
```

```
Cadena cola::primero()
{
  return Inicio->Datos;
}

bool cola::esvacia()
{
  return Inicio==NULL;
}

int cola::longitud()
{
  return ne;
}
```

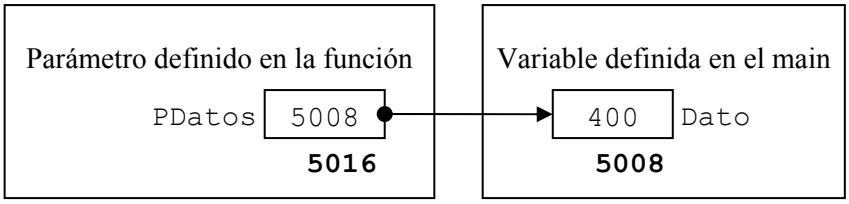
4. Paso de parámetros por puntero.

4.1 Paso de punteros a funciones o métodos.

Permite pasar al código de una función o método el contenido de una variable u objeto original, pasándole la dirección de la misma/o.

Para indicar el paso de punteros hay que declarar parámetros punteros. En la llamada hay que utilizar el operador **&** y el operador ***** o el operador **->** dentro del código de la función o método.

En el paso de parámetros solo se copia el primer nivel de la estructuras de datos.



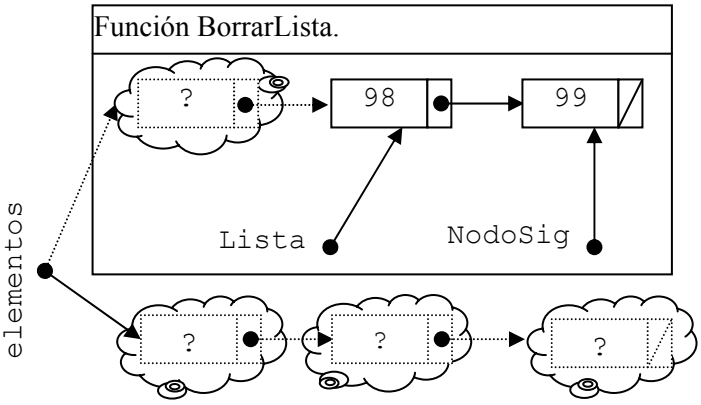

```
void Multiplica(float *PDatos) {
   *PDatos=*PDatos*2;
};

int main() {
  float Dato=400;
  //Cuando Termine Dato tendrá 800;
  Multiplica(&Dato);
  return 0;
}
```

Ejemplo 43. *Paso de parámetros por puntero.*

El paso de tablas a funciones o métodos siempre se realiza pasando el puntero que apunta el primer elemento de la tabla.

El paso por puntero en ocasiones puede tener un pequeño inconveniente. Supongamos que tenemos la función **borrarlista** que libera todos los nodos de una lista pasada por puntero. La llamada `borrarlista(elementos)` tiene el inconveniente de mantener el puntero original apuntado a la misma zona de memoria que ya se ha liberado y que no nos pertenece.

Una posible solución sería realizar después de la llamada una inicialización del puntero de la lista (`elementos = NULL`), pero esta solución delegaría al programador la responsabilidad de inicializar el puntero y un olvido supondría un error inexplicable en un breve espacio de tiempo.




```
void borrarlista(TNodoLista *Lista)
{
  TNodoLista *NodoSig;
  while(Lista!=NULL)   {
    NodoSig=Lista->Siguiente;
    delete Lista;
    Lista=NodoSig;
  };

}
```

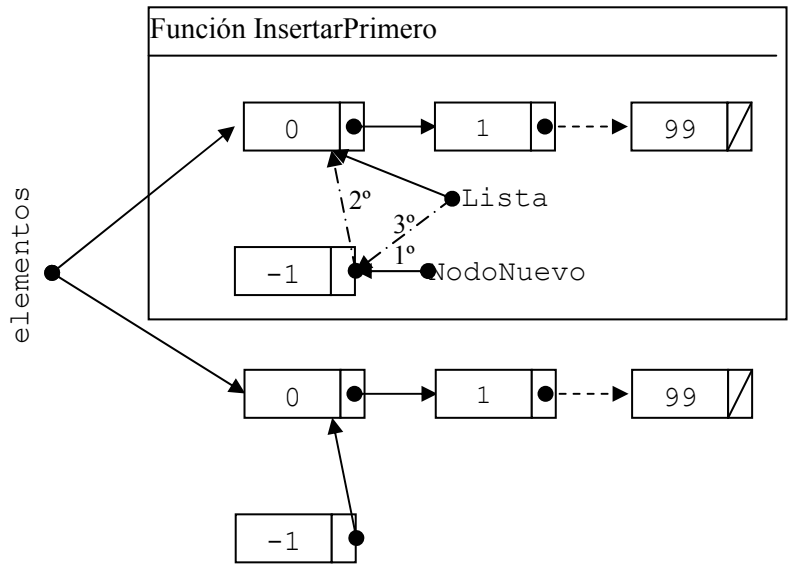
Ejemplo 44. *Eliminación incorrecta de una lista enlazada mediante un paso por puntero.*

Un error más grave surge en el proceso contrario es decir, cuando queremos insertar un nodo al principio de la lista.

El siguiente código inserta un elemento al principio de la lista pero realmente no lo hace produciendo a corto plazo un error de ejecución.

Dentro de la función, el nuevo nodo es insertado al principio pero cuando termina, tanto los parámetros como las variables locales desaparecen quedando la estructura de datos en un estado erróneo.

No solamente el nuevo nodo no es insertado sino que la memoria reservada, jamás podrá ser liberada debido a que no existe ningún puntero que apunte hacia ella.



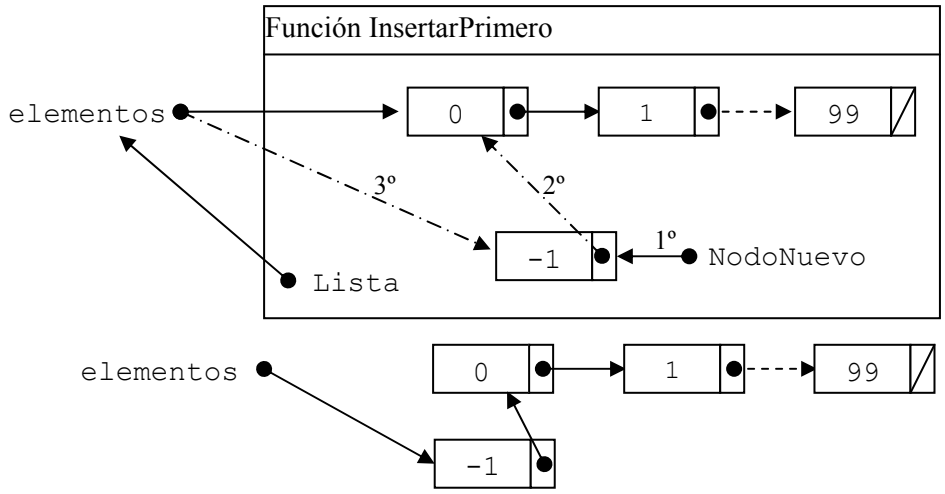

```
void InsertarPrimero(TNodoLista *Lista,
                     Tipo_Elemento valor)
{
  TNodoLista *NodoNuevo=new TNodoLista;  //P.1°
  if (NodoNuevo!=NULL)
  {
    NodoNuevo->Datos=valor;
    NodoNuevo->Siguiente=Lista;     //Paso 2°
    Lista=NodoNuevo;                //Paso 3°
  }
```

Ejemplo 45. *Inserción errónea de un nodo al principio de una lista mediante la utilización del paso de parámetros por puntero.*

Existen dos soluciones:

1. Utilizar un nodo cabecera. Es la más sencilla de implementar.
2. Utilizar punteros a punteros, es decir pasar la dirección de la variable puntero `elementos` a la función o método.

La llamada al método `InsertarPrimero` mediante la utilización de punteros a punteros sería `InsertarPrimero(&elementos,-1)` donde el operador **&** aplicado a una variable puntero, nos devuelve la dirección de memoria donde está situada la variable puntero, luego dicha dirección sería un puntero a un puntero.




```
void InsertarPrimero(TNodoLista **Lista, Tipo_Elemento valor)
{
  TNodoLista *NodoNuevo=new TNodoLista;  //Paso 1°
  if (NodoNuevo!=NULL)   {
    NodoNuevo->Datos=valor;
    NodoNuevo->Siguiente=*Lista;        //Paso 2°
    *Lista=NodoNuevo;                   //Paso 3°
  }
}
```

4.2 Paso de objetos a funciones o métodos.

- En la programación orientada a objetos, el paso de objetos a funciones o métodos puede producir el efecto colateral de eliminar la información estructurada de más de un nivel.

- El pase de un objeto a un método produce una copia exacta (a un primer nivel) del mismo objeto, compartiendo ambos objetos (original y copia) la memoria dinámica del objeto pasado.

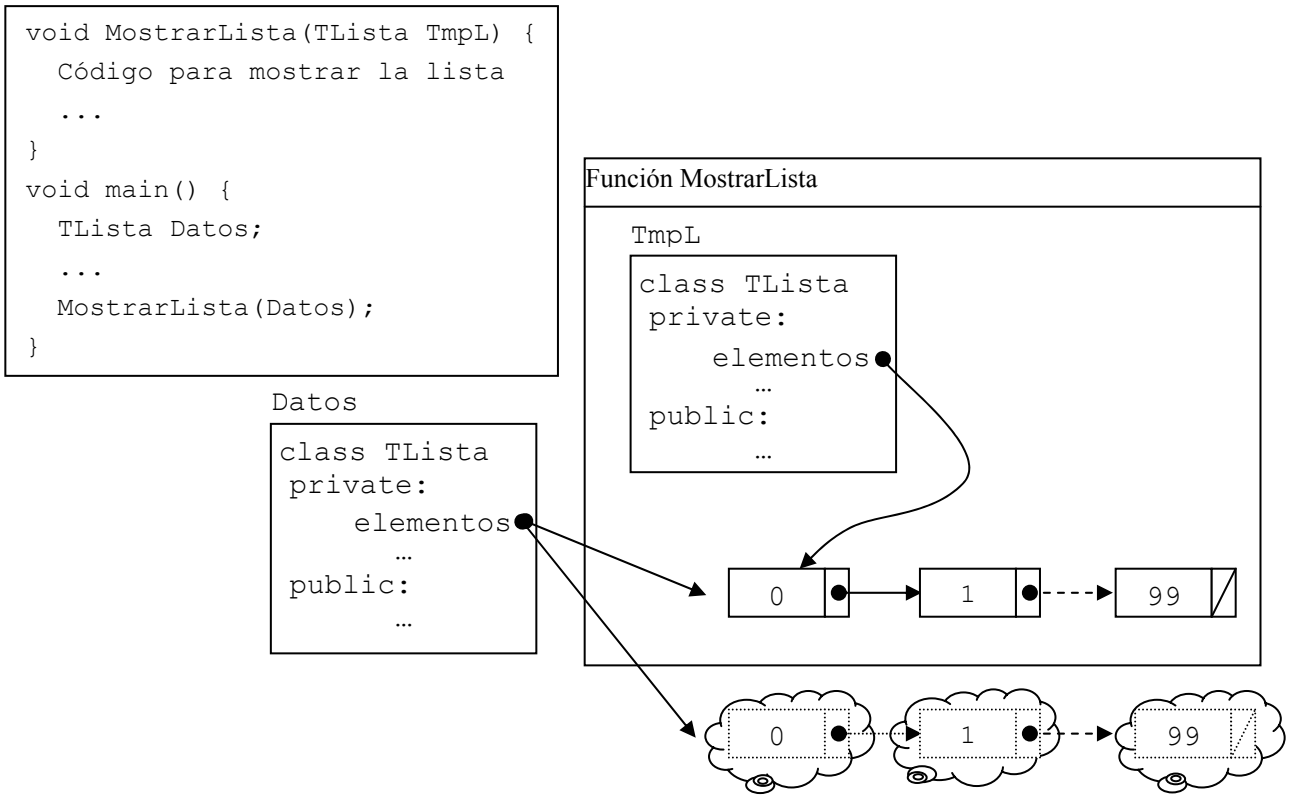
- Una vez terminado el método, el objeto parámetro temporal se destruye ejecutando su destructor que inevitablemente destruye la memoria dinámica compartida con el objeto original pasado por parámetro.

- El objeto original queda en un estado inconsistente al no conocer la eliminación de su memoria dinámica. Esto produce a muy corto plazo que el programa empiece a producir errores en ejecución. Para solucionar este problema se tienen tres soluciones:

    - Pasar la dirección del objeto original, es decir un puntero a un objeto.

    - Utilizar el paso por referencia.

    - La más sofisticada es la de crea un constructor denominado **constructor de copia** que se ejecuta cada vez que se pasa un objeto por copia a un método



Ejemplo 46. *Indeseable error producido por el destructor de la clase después del paso de un objeto a un método de la clase* `TLista`.

5. Introducción a las estructuras dinámicas no lineales.

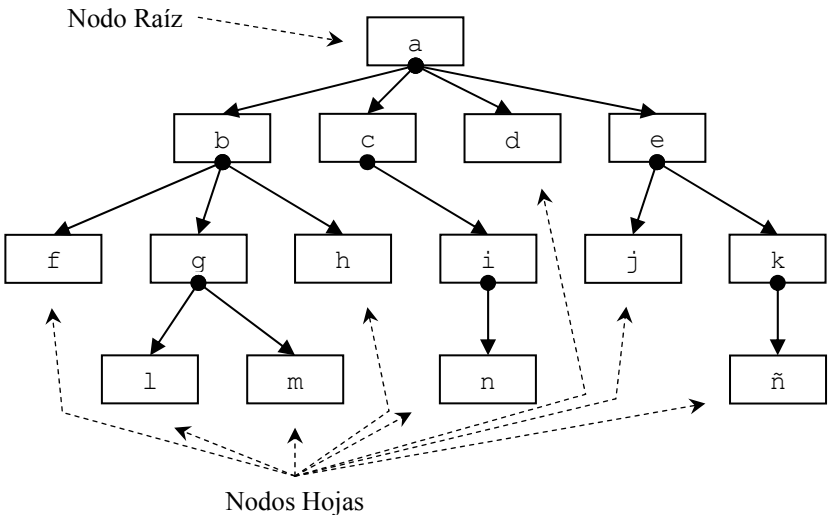
Las estructuras lineales de datos que como sabemos son aquellas cuyos elementos o nodos están dispuestos de manera secuencial verificando dos propiedades:

- Existen elementos distinguidos denominados *inicial* y *final* y que verifican que el elemento *inicial* no tiene otro elemento anterior y el elemento *final* verifica que no tiene un elemento después de él. Puede darse el caso que ambos elementos puedan ser el mismo.
- Cualquier otro elemento distinto a los elementos *inicial* y *final* verifica que tiene exactamente dos elementos uno anterior y otro posterior al el mismo.

Un ejemplo de estructuras de datos que no verifican las propiedades anteriores son por ejemplo las estructuras denominadas *Árboles* y las estructuras de datos denominadas *Grafos* que verifican:

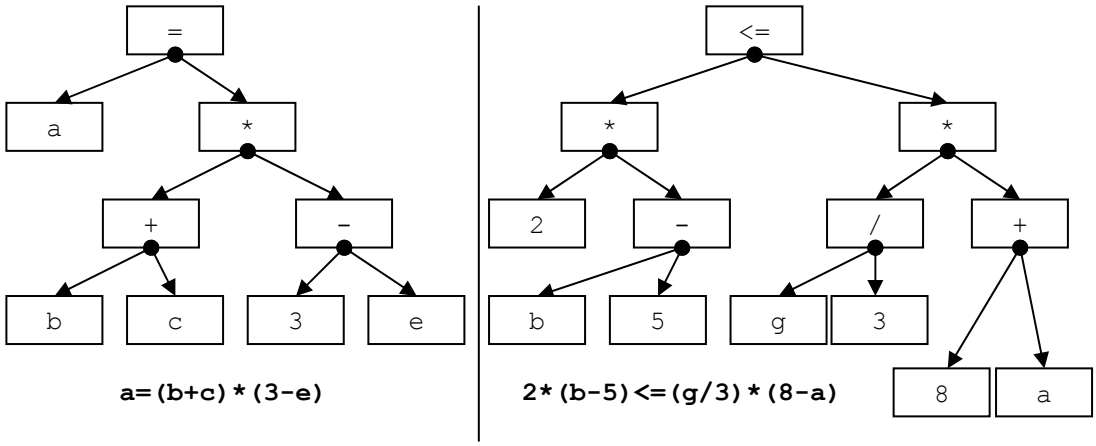
- **Árboles**: Son aquellas estructuras de datos que tienen como máximo un solo elemento anterior denominado Padre y un número variable de nodos siguientes denominados hijos.
- **Grafos**: Son aquellas estructuras de datos que tiene cualquier número de elementos o nodos anteriores y un número variable de nodos siguientes.

Un árbol se puede representar gráficamente de la siguiente manera:



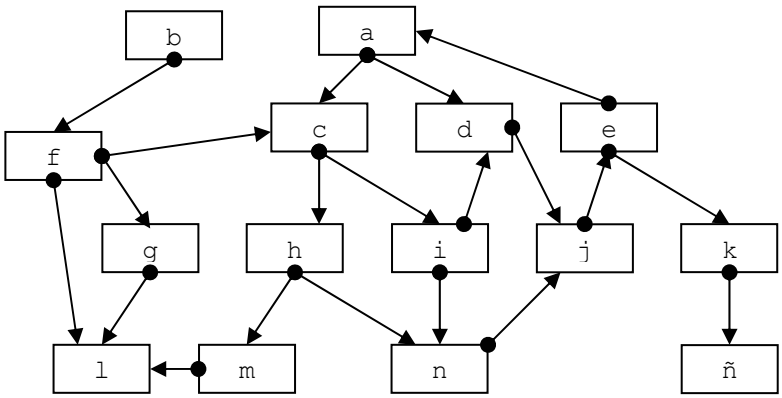
Los nodos en los árboles están relacionados según un orden **parental** de manera que dado un nodo (por ejemplo **b**), éste pueden tener un *padre* (nodo **a**) y un conjunto de nodos *hijos* (nodos **f**, **g**, **h**) de los cuales dos son nodos *hoja* (nodos **f** y **h**) y además un conjunto de nodos *hermanos* (nodos **c**, **d** y **e**).

Ejemplos de utilización:



Existen muchas clases de árboles con características particulares como pueden ser árboles binarios, balanceados, ordenados, tipo B, etc. En el caso de la figura anterior, son árboles binarios ya que siempre se verifica que dado un nodo el número de hijos, si es que tiene, es de solamente dos nodos.

Un grafo se puede representar gráficamente de la siguiente manera:



Como podemos ver en la figura anterior, dado un nodo determinado del grafo, éste puede tener más de un *padre* (nodo **n**, **c**), y existe más de un nodo que puede no tener padre (nodos **a** y **b**). En los grafos se presentan otras características que no aparecen en los árboles, estas son:

- Dado dos nodos del grafo puede existir más de un camino entre ambos, por ejemplo entre los nodos **b** y **l** se dan los caminos, **b-f-l**, **b-f-g-l**, y **b-f-c-h-m-l**.
- Dado un nodo puede existir un camino que vuelva a él mismo, es decir que exista un ciclo de nodos como por ejemplo **a-d-j-e-a**, **a-c-i-n-j-e-a**, y **a-c-h-n-j-e-a**.